



ASSESSMENT COVERSHEET

Attach this coversheet as the cover of your submission. All sections must be completed.

Section A: Submission Details

Programme : Bachelor in Information Technology (Honours) (Internet of Things)
Course Code & Name : IIB26603 MOBILE AND UBIQUITOUS COMPUTING
Course Lecturer(s) : Mohamad Adib Baihaqi bin Ahmad
Submission Title : Mini Project
Deadline : Day 20 Month June Year 2026 Time 12.00am
Penalties :

- 5% will be deducted per day to a maximum of four (4) working days, after which the submission will **not** be accepted.
- Plagiarised work is an Academic Offence in University Rules & Regulations and will be penalised accordingly.

Section B: Academic Integrity

Tick (✓) each box below if you agree:

- | | |
|-------------------------------------|--|
| ☒ | I have read and understood the UniKL's policy on Plagiarism in University Rules & Regulations. |
| ☒ | This submission is my own, unless indicated with proper referencing. |
| ☒ | This submission has not been previously submitted or published. |
| ☒ | This submission follows the requirements stated in the course. |

Section C: Submission Receipt

(must be filled in manually)

Office Receipt of Submission

Date & Time of Submission (stamp)	Student Name(s)	Student ID(s)
	Syahmi Bin Muhammad Shamsul	52224224168

Student Receipt of Submission

This is your submission receipt, the only accepted evidence that you have submitted your work. After this is stamped by the appointed staff & filled in, cut along the dotted lines above & retain this for your record.

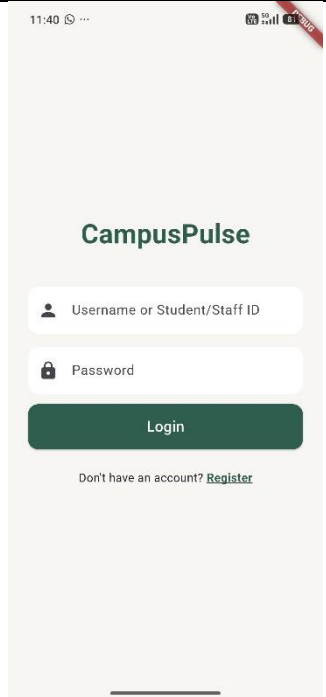
Date & Time of Submission (stamp)	Course Code	Submission Title	Student ID(s) & Signature(s)
	IIB22603	Mini Project: CampusPulse	52224224168

Project Overview

Project Title
CampusPulse
Project Description
CampusPulse is a mobile application developed using flutter SDK and by using SQLite database to manage users, bookings and admin functions. CampusPulse is designed for a campus environment where students/ staff can book campus facilities and admin can oversee users and bookings.

Output, Code Snippets & Explanation

1. login.dart

Output
 - Textfield = Username/ID, Password - Button = "Login" - GestureDetector = "Register"
Code Snippet
<pre>16 class _LoginPageState extends State<LoginPage> { 17 final TextEditingController idOrUsernameController = TextEditingController(); 18 final TextEditingController passwordController = TextEditingController(); 19 }</pre> Mutable Username and Password
<pre>27 if (input.isEmpty password.isEmpty) { 28 _showMessage("Please enter username/ID and password"); 29 return; 30 }</pre> Empty field validation
<pre>36 final user = users.firstWhere(37 (u) => 38 (u['username'] == input u['studentId'] == input) && 39 u['password'] == password, 40 orElse: () => {}, 41);</pre> Matching username/ID and password validation

```

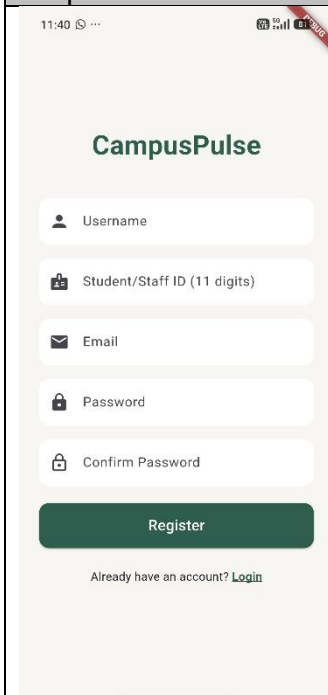
51     final role = user['role'];
52     final userId = user['id'];
53
54     if (!mounted) return;
55     Navigator.pushReplacement(
56       context,
57       MaterialPageRoute(builder: (_) => ColumnPage(role: role, userId: userId)),
58     );

```

If role admin, go to columnpage (admin). If public, go to columnpage (public).

2. register.dart

Output



- Textfield = Username, Student/staff ID (11 digits) , Email, Password (obscured), Confirm Password (match & obscured)
- Button = "Register"
- GestureDetector = "Login"

Code Snippet

```

15 class _RegisterPageState extends State<RegisterPage> {
16   final TextEditingController nameController = TextEditingController();
17   final TextEditingController studentIdController = TextEditingController();
18   final TextEditingController emailController = TextEditingController();
19   final TextEditingController passwordController = TextEditingController();
20   final TextEditingController confirmController = TextEditingController();

```

Mutable Username, ID, Email, Password, confirm password.

```

31   if (username.isEmpty || studentId.isEmpty || email.isEmpty || password.isEmpty || confirm.isEmpty) {
32     _showMessage("Please fill in all fields");
33     return;
34   }

```

Empty field validation

```

36   //-----Student/Staff ID validation (must be 11 digits)-----
37   if (studentId.length != 11 || int.tryParse(studentId) == null) {
38     _showMessage("Student/Staff ID must be exactly 11 digits");
39     return;
40   }

```

11 digits ID validation

```

43   if (password != confirm) {
44     _showMessage("Passwords do not match");
45     return;
46   }

```

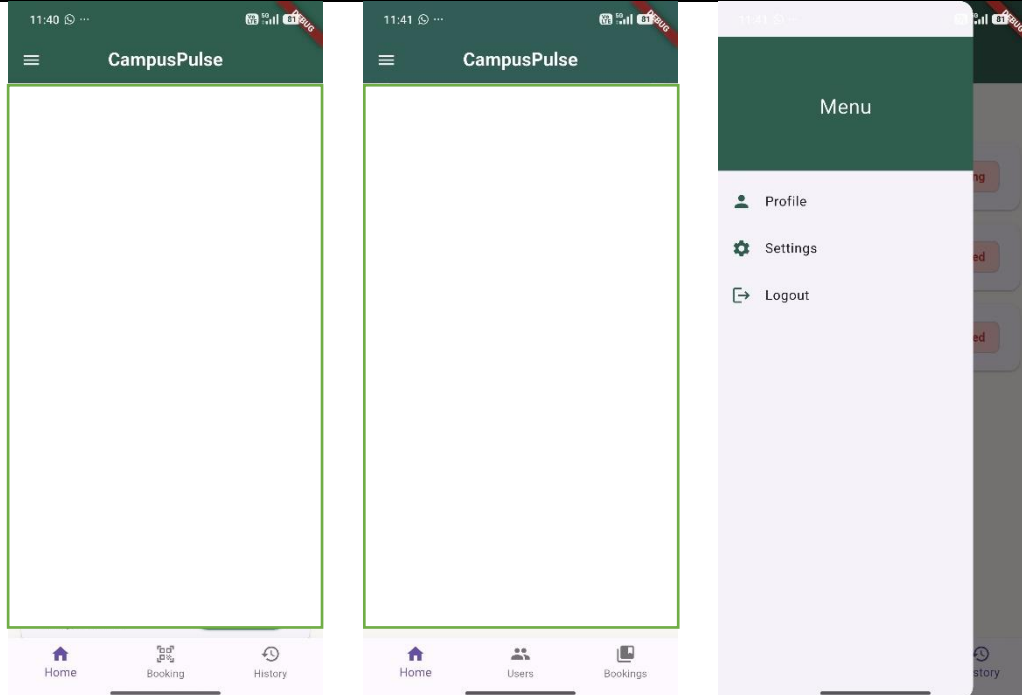
Matching confirm password with password validation

```
63     if (!mounted) return;
64     _showMessage("Registration successful!");
65     Navigator.pushReplacement(
66       context,
67       MaterialPageRoute(builder: (_) => const LoginPage()),
```

If register successful, go back to log in page.

3. column.dart

Output



BottomBarNavigation:

public = "Home", "Booking", "History"
admin = "Home", "Users", "Bookings"

Drawer = "Profile", "Setting", "Logout"

Code Snippet

```
31     final pages = widget.role == "Admin"
32     ? [
33       AdminHomePage(userId: widget.userId),
34       const ManageUserPage(),
35       const ManageBookingPage(),
36     ]
37     : [
38       PublicHomePage(userId: widget.userId),
39       BookingPage(userId: widget.userId),
40       HistoryPage(userId: widget.userId),
41     ];
```

List of pages for bottombarnavigation:

Admin = adminhomepage, manageuserpage, managebookingpage
Public = publichomepage, bookingpage, historypage

```

68     ListTile(
69       leading: const Icon(Icons.person, color: Color(0xFF2E5E4E)),
70       title: const Text("Profile"),
71       onTap: () {
72         Navigator.push(
73           context,
74           MaterialPageRoute(
75             builder: (context) => ProfilePage(userId: widget.userId),
76           ), // MaterialPageRoute
77         );
78       },
79     ), // ListTile
80     ListTile(
81       leading: const Icon(Icons.settings, color: Color(0xFF2E5E4E)),
82       title: const Text("Settings"),
83       onTap: () {
84         Navigator.push(
85           context,
86           MaterialPageRoute(builder: (context) => const SettingsPage()),
87         );
88       },
89     ), // ListTile
90     ListTile(
91       leading: const Icon(Icons.logout, color: Color(0xFF2E5E4E)),
92       title: const Text("Logout"),
93       onTap: () {
94         Navigator.pushReplacement(
95           context,
96           MaterialPageRoute(builder: (context) => const LoginPage()),
97         );
98       },
99     ), // ListTile

```

Sidebar drawer listtile:
 Profile => profilepage
 Settings => settingspage
 Logout => loginpage

```

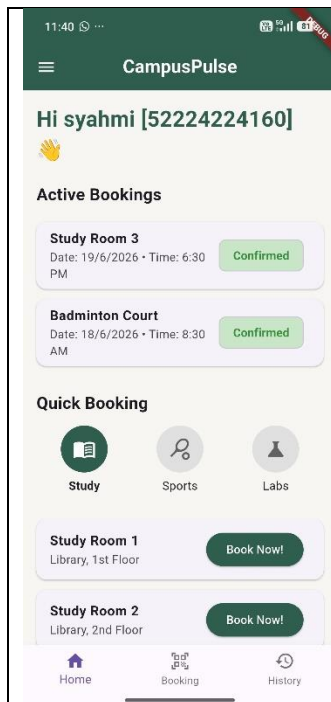
103     body: pages[currentIndex],
104     bottomNavigationBar: BottomNavigationBar(
105       currentIndex: currentIndex,
106       onTap: (index) => setState(() => currentIndex = index),
107       items: widget.role == "Admin"
108         ? const [
109           BottomNavigationBarItem(icon: Icon(Icons.home), label: "Home"),
110           BottomNavigationBarItem(icon: Icon(Icons.people), label: "Users"),
111           BottomNavigationBarItem(icon: Icon(Icons.collections_bookmark), label: "Bookings"),
112         ]
113         : const [
114           BottomNavigationBarItem(icon: Icon(Icons.home), label: "Home"),
115           BottomNavigationBarItem(icon: Icon(Icons.qr_code_scanner), label: "Booking"),
116           BottomNavigationBarItem(icon: Icon(Icons.history), label: "History"),
117         ],
118     ), // BottomNavigationBar

```

Bottombarnavigation set as current index = 0

4. publichome.dart

Output



- Greetings + Staff/studentID
- Active booking section – shows approved booking
- Quick booking section – filter places by category
- Button – “Book Now!” => bookingpage

Code Snippet

```

22 final Map<String, List<Map<String, String>>> spaces = {
23     "Study": [
24         {"title": "Study Room 1", "subtitle": "Library, 1st Floor"},
25         {"title": "Study Room 2", "subtitle": "Library, 2nd Floor"},
26         {"title": "Study Room 3", "subtitle": "Library, 3rd Floor"},
27     ],
28     "Sports": [
29         {"title": "Badminton Court", "subtitle": "Sports Complex"},
30         {"title": "Volleyball Court", "subtitle": "Sports Complex"},
31         {"title": "Pickleball Court", "subtitle": "Sports Complex"},
32     ],
33     "Labs": [
34         {"title": "Chemistry Lab", "subtitle": "Science Building"},
35         {"title": "Electrical Lab", "subtitle": "Engineering Wing"},
36         {"title": "Computer Lab", "subtitle": "Innovation Hub"},
37     ],
38 };

```

List of campus facilities

```

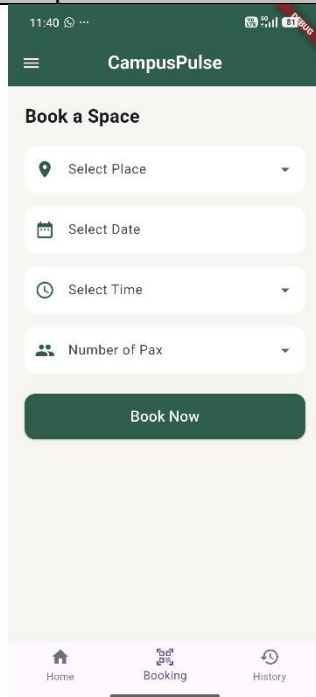
135 Row(
136   mainAxisAlignment: MainAxisAlignment.spaceAround,
137   children: [
138     _CategoryCard(
139       icon: Icons.menu_book,
140       label: "Study",
141       isSelected: selectedCategory == "Study",
142       onTap: () => setState(() => selectedCategory = "Study"),
143     ), // _CategoryCard
144     _CategoryCard(
145       icon: Icons.sports_tennis,
146       label: "Sports",
147       isSelected: selectedCategory == "Sports",
148       onTap: () => setState(() => selectedCategory = "Sports"),
149     ), // _CategoryCard
150     _CategoryCard(
151       icon: Icons.science,
152       label: "Labs",
153       isSelected: selectedCategory == "Labs",
154       onTap: () => setState(() => selectedCategory = "Labs"),
155     ), // _CategoryCard
156   ],
157 ), // Row

```

Filter for campus facilities by category

5. booking.dart

Output



Dropdown menu:

- place
- time
- pax

Datepicker = date

Button = "Book Now"

Code Snippet

```

23 final List<String> places = [
24   "Study Room 1",
25   "Study Room 2",
26   "Study Room 3",
27   "Badminton Court",
28   "Volleyball Court",
29   "Pickleball Court",
30   "Chemistry Lab",
31   "Electrical Lab",
32   "Computer Lab",
33 ];

```

List for places/facilities

```

35   final List<String> timeSlots = [
36     "8:30 AM", "9:30 AM", "10:30 AM", "11:30 AM", "12:30 PM", "1:30 PM", "2:30 PM",
37     "3:30 PM", "4:30 PM", "5:30 PM", "6:30 PM", "7:30 PM", "8:30 PM", "9:30 PM", "10:30 PM"
38   ];

```

List for time

```

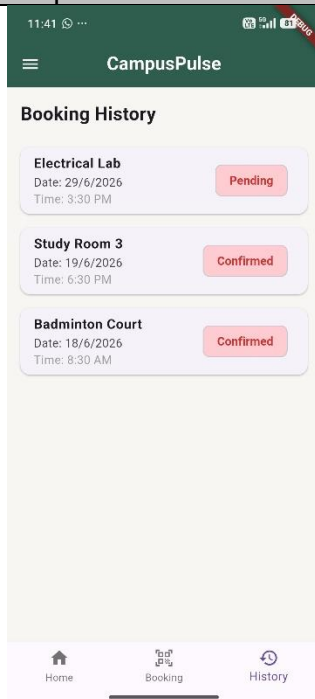
40   final List<String> paxOptions = ["Single", "Pair", "Group"];

```

List for pax

6. history.dart

Output



-Shows all booking made (confirmed/cancelled/pending)

Code Snippet

```

21   Future<void> _loadBookings() async {
22     final db = DatabaseHelper.instance;
23     final allBookings = await db.getAllBookings();
24
25     if (!mounted) return;
26     setState(() {
27       // Filter bookings by logged-in user
28       pastBookings = allBookings.where((b) => b['userId'] == widget.userId).toList();
29     });
30   }

```

Display all booking from database

7. profile.dart

Output

- CircleAvatar – can be place with images
- Card = ID, Email, Phone, Course, Semester
- Button = “Edit Profile” => edit profile pop up form

Code Snippet

8. settings.dart

- switch = enable notifications, dark mode
- Card = edit profile, change password, logout

Code Snippet

9. adminhome.dart

Output

- Greetings + student/staff ID
- Dashboard overview – shows total bookings, total active users, total pending approvals.
- Booking statistics – shows booking chart over the year (future implementation)
- Recent activity – shows 5 latest bookings
- GestureDetector – “View All Bookings” => Bookinghistorypage

Code Snippet

10. manageuser.dart

Output

- card = Existing users
- Dropdown menu:

Role = change user role to admin/ user (public)
Status = change user status to valid/ ban

Code Snippet

11. managebooking.dart

Output



- card = Booking details
- Button = "Approve" confirmed the booking, "Cancel" cancel the booking
- Search Bar = search for specific booking.

Code Snippet

12. bookinghistory.dart

Output



Search Bar – search specific booking by ID, place, date or time
Dropdown menu – filter booking by status
(Confirmed/Pending/Cancelled)

Card – Bookings details

Code Snippet

DATABASE SQLite

Filename: database_helper.dart

```
12 Future<Database> get database async {  
13   if (_database != null) return _database!;  
14   _database = await _initDB('campuspulse.db');  
15   return _database!;  
16 }
```

If campuspulse.db not existed, create new one

```
26 Future _createDB(Database db, int version) async {  
27   //-----create table user-----  
28   await db.execute('''  
29     CREATE TABLE user (  
30       id INTEGER PRIMARY KEY AUTOINCREMENT,  
31       username TEXT NOT NULL,  
32       studentId TEXT UNIQUE NOT NULL CHECK(length(studentId) = 11),  
33       email TEXT UNIQUE NOT NULL,  
34       password TEXT NOT NULL,  
35       role TEXT NOT NULL DEFAULT 'User',  
36       status TEXT NOT NULL DEFAULT 'Valid',  
37       phone TEXT,  
38       course TEXT,  
39       semester TEXT  
40     )  
41     ''');
```

If user Table did not exist, create new one.

```
44   await db.execute('''  
45     CREATE TABLE booking (  
46       id INTEGER PRIMARY KEY AUTOINCREMENT,  
47       userId INTEGER,  
48       date TEXT,  
49       time TEXT,  
50       place TEXT,  
51       status TEXT NOT NULL DEFAULT 'Pending',  
52       FOREIGN KEY (userId) REFERENCES user (id) ON DELETE CASCADE  
53     )  
54     ''');
```

If booking table did not exist, create new one.

```
57   await db.insert('user', {  
58     'username': 'admin',  
59     'studentId': '52224224168',  
60     'email': 'admin@example.com',  
61     'password': '12345',  
62     'role': 'Admin',  
63     'status': 'Valid',  
64     'phone': '',  
65     'course': '',  
66     'semester': ''  
67   }, conflictAlgorithm: ConflictAlgorithm.ignore);  
68 }
```

Create default admin. If admin already created, ignore this query.

```

71 Future<int> insertUser(Map<String, dynamic> row) async {
72     final db = await instance.database;
73     return await db.insert('user', row);
74 }
75
76 Future<List<Map<String, dynamic>>> getAllUsers() async {
77     final db = await instance.database;
78     return await db.query('user');
79 }
80
81 Future<int> updateUserRole(int id, String role) async {
82     final db = await instance.database;
83     return await db.update('user', {'role': role}, where: 'id = ?', whereArgs: [id]);
84 }
85
86 Future<int> updateUserStatus(int id, String status) async {
87     final db = await instance.database;
88     return await db.update('user', {'status': status}, where: 'id = ?', whereArgs: [id]);
89 }
90
91 Future<int> updateUserProfile(int id, Map<String, dynamic> fields) async {
92     final db = await instance.database;
93     return await db.update('user', fields, where: 'id = ?', whereArgs: [id]);
94 }
95

```

Table user functions:

InsertUser = insert new user into database

getAllUser = display all user

updateUserRole = change user role (admin/user)

updateUserStatus = change user status (valid/ban)

updateUserProfile = change user profile

```

97 Future<int> insertBooking(Map<String, dynamic> row) async {
98     final db = await instance.database;
99     return await db.insert('booking', row);
100 }
101
102 Future<List<Map<String, dynamic>>> getAllBookings() async {
103     final db = await instance.database;
104     return await db.query('booking', orderBy: 'id DESC');
105 }
106
107 Future<int> updateBookingStatus(int id, String status) async {
108     final db = await instance.database;
109     return await db.update('booking', {'status': status}, where: 'id = ?', whereArgs: [id]);
110 }
111
112 Future<int> deleteBooking(int id) async {
113     final db = await instance.database;
114     return await db.delete('booking', where: 'id = ?', whereArgs: [id]);
115 }

```

Table booking functions:

insertBooking = insert new booking into database

getAllBookings= display all bookings

updateBookingStatus = change booking status (pending/confirmed/cancel)

deletebooking = delete the booking